UNIVERSITAS MULTI DATA PALEMBANG
Fakultas Ilmu Komputer dan Rekayasa
Program Studi Sistem Informasi

TEMPLAT LAPORAN

TUGAS 2

Postman Collection dan Automated Test Script

Pengujian *RESTful API* dengan *Postman Automation*

Pengembangan Aplikasi Web II | SUB-CPMK-7

Nama dan NIM	M.Raihan Al Lutfi (2428240156)
Kelas	SI5C
Judul project / topik API	Area Parking
Tautan repositori	raihannlutfi/Tugas1-Resful-2428240156
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	16-10-2026

Petunjuk Pengisian

Dokumen ini adalah laporan **Tugas 2**: menguji RESTful API project akhir Anda dengan Postman. Seluruh isinya menggunakan **API milik Anda sendiri**, bukan API Mahasiswa pada contoh materi. Pelajari langkahnya pada materi Pertemuan 8 sebelum mengisi.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya `[sumber-daya]`, dengan data API Anda.
2. Bukti berupa gambar (*screenshot*) diisi pada kotak bergaris putus-putus. Cuplikan kode test script **ditempel sebagai teks**, bukan gambar.
3. Setiap bukti memiliki keterangan dan kotak **Penjelasan**. Tulis penjelasan dengan kalimat Anda sendiri dan sebut nama *request* atau fungsi dari API Anda.
4. Simpan hasil akhir sebagai PDF, lalu kumpulkan melalui LMS sesuai ketentuan dosen (format nama berkas dan batas waktu mengikuti petunjuk di LMS).

B. Bukti yang diminta

No	Bukti	Bagian	Jenis
1	Daftar request dalam Collection	Bab 2	Screenshot (wajib)
2	Cuplikan test script	Bab 3	Kode teks (wajib)
3	Ringkasan Collection Runner	Bab 4	Screenshot (wajib)
4	Hasil Newman di terminal	Bab 5	Screenshot (pengayaan)

C. Syarat tangkapan layar

Bukti	Yang harus terlihat
Collection	Nama Collection, kedua folder (bila ada), dan seluruh nama request. Buka semua folder sebelum mengambil gambar.
Ringkasan Runner	Nama Collection, Environment yang dipakai, jumlah request, jumlah test **lulus** dan **gagal**. Seluruh test harus lulus (0 gagal).
Newman	Perintah yang diketik, daftar hasil, dan tabel `executed` dan `failed` di bagian akhir.
Larangan	Gambar dipotong sehingga angka hasil tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

> **Sebelum mengambil screenshot Runner**
> Nyalakan ulang server agar data kembali ke kondisi awal, pilih Environment yang benar, lalu jalankan seluruh Collection. Bila ada test gagal, perbaiki dulu dan jalankan ulang sampai 0 gagal.

Bab 1 Identitas API dan Daftar Endpoint

Bab ini menjelaskan API yang diuji, agar pembaca memahami cakupan pengujian pada bab berikutnya.

1.1 Deskripsi singkat API

Topik API saya adalah pengelolaan data parkir kendaraan (*parking-record*). Entitas utamanya adalah data parkir kendaraan yang berisi nomor plat, jenis kendaraan, waktu masuk, waktu keluar, dan biaya parkir. Teknologi yang digunakan adalah Express.js dengan struktur Route, Controller, dan Model untuk mengatur alur pengelolaan data.

1.2 Daftar endpoint

Tuliskan **seluruh** *endpoint* API Anda. Kolom "API key" diisi Ya bila rute dilindungi `cekApiKey` atau mekanisme serupa.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	{{base_url}}/parking-records	Tidak	200	Menampilkan semua data catatan parkir.
2	GET	{{base_url}}/parking-records/{{parkingrecordsModelID}}	Tidak	200	Menampilkan satu data parkir berdasarkan ID.
3	POST	{{base_url}}/parking-records	YA	201	Menambahkan data parkir baru.
4	PUT	{{base_url}}/parking-records/{{parkingrecordsModelID}}	YA	200	Mengubah data parkir berdasarkan ID.
5	DELETE	{{base_url}}/parking-records/{{parkingrecordsModelID}}	YA	204	Menghapus data parkir berdasarkan ID.
6					
7					
8					

1.3 Variabel Environment

Tuliskan variabel yang Anda simpan di Environment. Jangan menuliskan nilai rahasia sungguhan.

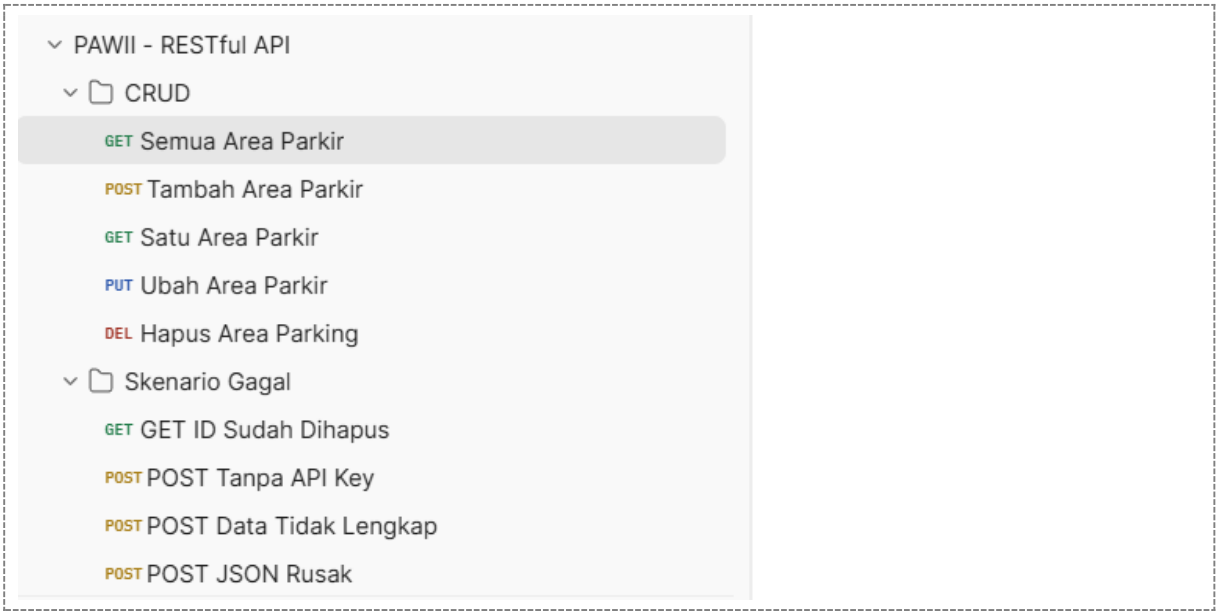
No	Nama variabel	Nilai (contoh)	Fungsi
1	base_url	http://localhost:3000	Alamat server
2	api_key	isi-dengan-kunci-anda	Nilai header kunci API
3	parkingRecordsModelID	(kosong, diisi script)	ID dari respons POST
4			

Bab 2 Postman Collection

Bab ini membuktikan bahwa seluruh *endpoint* sudah dikelompokkan dalam satu Collection.

2.1 Daftar request dalam Collection

Harus terlihat pada screenshot: nama Collection dan seluruh request di dalam folder. Buka semua folder.



Gambar 1. Collection beserta seluruh request

Penjelasan:
Saya mengelompokkan request menjadi dua folder, yaitu CRUD dan Skenario Gagal supaya lebih mudah dipahami dan diuji. Folder CRUD berisi request untuk melihat semua data area parkir, menambah data, melihat satu data, mengubah data, dan menghapus data. Saya meletakkan operasi ini terlebih dahulu karena merupakan fungsi utama dari API yang saya buat. Setelah itu, folder Skenario Gagal digunakan untuk menguji kondisi ketika terjadi kesalahan, seperti mengambil ID yang sudah dihapus, menambahkan data tanpa API Key, mengirim data yang tidak lengkap, dan mengirim JSON yang rusak. Dengan pembagian ini, saya bisa lebih mudah memeriksa apakah API berjalan dengan benar, termasuk saat menerima request yang salah.

2.2 Tabel request

Salin daftar request pada gambar di atas ke tabel berikut. Setiap *endpoint* pada Bab 1 harus punya minimal satu request.

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
1	Semua Area Parkir	GET	{{base_url}}/parking-records	CRUD
2	Tambah Area Parkir	POST	{{base_url}}/parking-records	CRUD

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
3	Satu Area Parkir	GET	{{base_url}}/parking-records/{{parkingRecordsModelID}}	CRUD
4	Ubah Area Parkir	PUT	{{base_url}}/parking-records/{{parkingRecordsModelID}}	CRUD
5	Hapus Area Parkir	DELETE	{{base_url}}/parking-records/{{parkingRecordsModelID}}	CRUD
6	GET ID Sudah Dihapus	GET	{{base_url}}/parking-records/{{parkingRecordsModelID}}	Skenario Gagal
7	POST Tanpa API Key	POST	{{base_url}}/parking-records	Skenario Gagal
8	POST Data Tidak Lengkap	POST	{{base_url}}/parking-records	Skenario Gagal
9	POST JSON Rusak	POST	{{base_url}}/parking-records	Skenario Gagal
10				

Bab 3 Automated Test Script

Bab ini menunjukkan *assertion* yang Anda tulis. Setiap request wajib memiliki minimal satu *assertion* status code dan satu *assertion* struktur atau isi data JSON.

3.1 Rekap assertion per request

No	Nama request	Assertion status code	Assertion struktur atau isi JSON
1	Semua Area Parkir	Status code adalah 200	"id": 1, "platNomor": "BG 1234 AA", "jenisKendaraan": "motor"
2	Tambah Area Parkir	Status code adalah 201	"id": 3, "platNomor": "BG 1234 AA", "jenisKendaraan": "motor"
3	Satu Area Parkir	Status code adalah 200	"id": 1, "platNomor": "BG 1234 AA"
4	Ubah Area Parkir	Status code adalah 200	id": 3,"platNomor": "BG 1111 AB", "jenisKendaraan": "Motor",
5	Hapus Area Parkir	Status code adalah 204	Tidak ada isi JSON jika respons menggunakan status 204
6	GET ID Sudah Dihapus	Status code adalah 404	JSON berisi pesan data tidak ditemukan
7	POST Tanpa API Key	Status code adalah 401	JSON berisi pesan API Key tidak valid
8	POST Data Tidak Lengkap	Status code adalah 400	JSON berisi pesan platNomor dan jenisKendaraan wajib diisi
9	POST JSON Rusak	Status code adalah 400	JSON berisi pesan format JSON tidak valid
10			

3.2 Cuplikan 1: request GET

Tempel test script **satu request GET** pilihan Anda.

```
// Test script request GET: [Semua Area Parkir]

pm.test('Status code adalah 200', function () {
  pm.response.to.have.status(200);
});

pm.test('Data area parkir memiliki struktur yang sesuai', function () {
  const jsonData = pm.response.json();

  pm.expect(jsonData).to.be.an('array');

  if (jsonData.length > 0) {
    pm.expect(jsonData[0]).to.have.property('id');
    pm.expect(jsonData[0]).to.have.property('platNomor');
    pm.expect(jsonData[0]).to.have.property('jenisKendaraan');
    pm.expect(jsonData[0]).to.have.property('waktuMasuk');
    pm.expect(jsonData[0]).to.have.property('waktuKeluar');
    pm.expect(jsonData[0]).to.have.property('biaya');
  }
});
```

Penjelasan:

Penjelasan Fungsi Setiap pm.test

Pada pengujian pertama, pm.test('Status code adalah 200') digunakan untuk memeriksa apakah request GET berhasil dan menghasilkan status code 200. Pemeriksaan ini penting untuk memastikan server bisa memberikan respons dengan benar.

Pada pengujian kedua, pm.test('Data area parkir memiliki struktur yang sesuai') digunakan untuk memeriksa apakah respons berbentuk array dan data parkir memiliki field id, platNomor, jenisKendaraan, waktuMasuk, waktuKeluar, dan biaya. Pemeriksaan ini penting supaya data yang diterima sesuai dengan struktur yang dibutuhkan oleh API Area Parkir.

3.3 Cuplikan 2: request POST atau request yang menyimpan variabel

Tempel test script **request POST** (atau request lain yang menyimpan variabel ke Environment).

```
// Test script request POST: [Tambah Area Parkir]
pm.test('Status code adalah 201', function () {
  pm.response.to.have.status(201);
});

pm.test('Data parkir memiliki id dan plat nomor sesuai kiriman', function () {
  const jsonData = pm.response.json();
  pm.expect(jsonData).to.have.property('id');
  pm.expect(jsonData.platNomor).to.eql('BG 1234 AA');
  pm.expect(jsonData.jenisKendaraan).to.eql('motor');
});

if (pm.response.code === 201) {
  pm.environment.set('parkingrecordsModelID', pm.response.json().id);
}
```

Penjelasan:

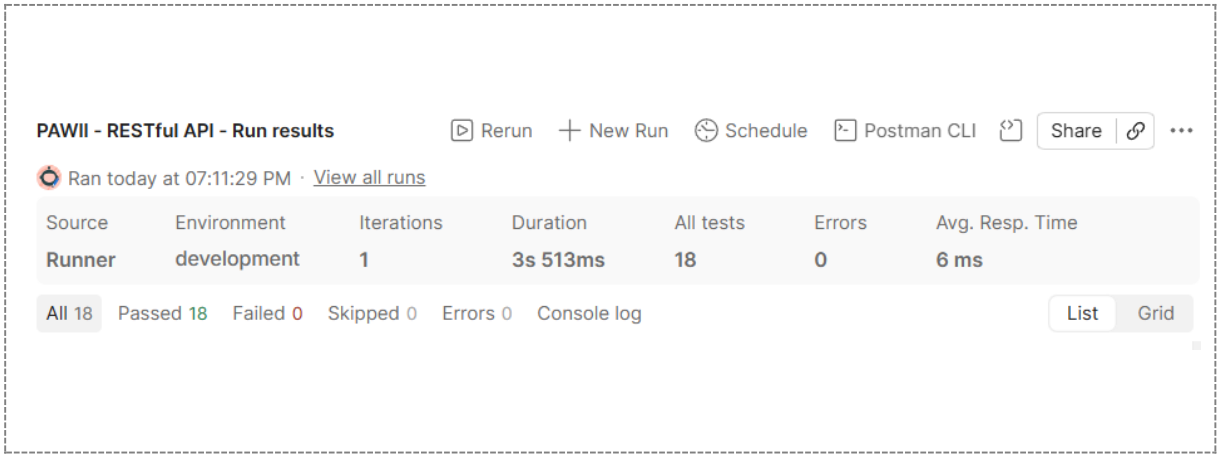
Data dari respons request POST Tambah Area Parkir disimpan ke environment Postman menggunakan variabel parkingrecordsModelID. Data yang disimpan adalah ID dari catatan parkir yang berhasil ditambahkan. Variabel ini kemudian digunakan oleh request GET Satu Area Parkir, PUT Ubah Area Parkir, dan DELETE Hapus Area Parkir untuk menentukan data yang akan diakses, diubah, atau dihapus. Urutan request perlu diperhatikan karena POST harus dijalankan terlebih dahulu agar ID tersedia. Jika request GET, PUT, atau DELETE dijalankan sebelum POST, ID bisa saja belum tersedia atau masih menggunakan ID lama sehingga pengujian tidak berjalan sesuai harapan.

Bab 4 Hasil Collection Runner

Bab ini adalah bukti utama bahwa seluruh test lolos saat Collection dijalankan sekaligus.

4.1 Ringkasan Collection Runner

Harus terlihat pada screenshot: nama Collection, Environment, jumlah request, jumlah test lulus dan gagal. Seluruh test harus lulus.



Gambar 2. Ringkasan hasil Collection Runner

Penjelasan:
Berdasarkan hasil pengujian di Postman, saya menjalankan 9 request dengan total 18 test. Semua test berhasil lulus dengan hasil Passed 18, Failed 0, dan Errors 0. Sebelum menjalankan Runner, saya memastikan server API berjalan, environment development sudah dipilih, dan urutan request sudah benar agar pengujian dapat berjalan dengan baik.

4.2 Rekap angka hasil

Nama Collection	PAWII - RESTful API
Environment yang dipakai	Development
Jumlah request dijalankan	9 request
Jumlah test lulus	18
Jumlah test gagal	0
Tanggal dan jam pengujian	16/10/2026 19:18

4.3 Temuan selama pengujian

Catat kegagalan yang pernah muncul sebelum akhirnya lulus. Bagian ini menunjukkan bahwa pengujian benar-benar membantu menemukan masalah. Bila tidak ada, tulis "tidak ada kegagalan".

No	Request	Kegagalan (pesan)	Penyebab	Perbaikan
1	GET Satu Area Parkir	Mendapat status 404 Not Found	ID yang digunakan tidak ditemukan dalam data catatan parkir.	Menggunakan ID yang benar dan memastikan data tersedia sebelum menjalankan request.
2	PUT Ubah Area Parkir	Mendapat pesan Rute PUT /parking-records/ tidak ditemukan	URL tidak menyertakan ID data parkir di bagian akhir.	Memperbaiki URL menjadi {{base_url}}/parking-records/{{parkingrecordsModelID}} dan memastikan variabel ID sudah terisi.

Bab 5 Newman di Terminal (Pengayaan)

> **Bagian ini opsional**
> Bab ini adalah pengayaan dan tidak mengubah kriteria penilaian. Isi bila Anda menjalankan Collection dengan Newman. Bila tidak, tulis "tidak dikerjakan" di bawah judul ini.

5.1 Hasil Newman

Harus terlihat pada screenshot: perintah `newman run ...`, daftar hasil, dan tabel `executed` dan `failed`.

[Tempel screenshot di sini]

Gambar 3. Hasil menjalankan Collection dengan Newman

Gambar 3. Hasil menjalankan Collection dengan Newman

Penjelasan:

Tulis perintah yang Anda jalankan dan jelaskan arti exit code (0 atau 1) pada hasil Anda.

Bab 6 Repositori dan Berkas

Tautan repositori	raihannlutfi/Tugas1-Resful-2428240156
Lokasi berkas Collection (.json)	postman/PAWII - RESTful API.postman_collection.json
Lokasi berkas Environment (.json)	postman/development.postman_environment.json
Berkas Environment tidak memuat kunci rahasia sungguhan	[✅] Ya [ ] Tidak
Repositori dapat dibuka dosen	[✅] Ya [ ] Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan pengalaman Anda mengerjakan tugas ini.

Pertanyaan 1

Dari seluruh *assertion* yang Anda tulis, mana yang paling berguna untuk menemukan kesalahan pada API Anda? Berikan alasannya.

Penjelasan:
Menurut saya, assertion yang paling berguna adalah memeriksa status code dan isi respons dari API. Alasannya, kedua pemeriksaan ini membantu mengetahui apakah request berhasil dijalankan dan apakah data yang dikembalikan sudah sesuai dengan yang diharapkan. Contohnya, status 201 menunjukkan data berhasil ditambahkan, sedangkan status 404 menunjukkan data atau rute yang diminta tidak ditemukan. Dengan begitu, kesalahan pada API lebih mudah diketahui dan diperbaiki.

Pertanyaan 2

Apa perbedaan yang Anda rasakan antara menguji API secara manual dan memakai Collection Runner? Bagian mana yang paling sulit saat menulis test script, dan bagaimana Anda mengatasinya?

Penjelasan:
Menurut saya, pengujian API secara manual lebih mudah untuk memeriksa satu request karena saya bisa langsung melihat hasil responsnya. Sedangkan dengan Collection Runner, saya bisa menjalankan banyak request sekaligus sehingga lebih hemat waktu dan hasil pengujiannya lebih mudah diperiksa.
Bagian yang paling sulit saat menulis test script adalah membuat pengujian yang sesuai dengan hasil respons API dan menyimpan ID data agar bisa digunakan pada request berikutnya. Saya mengatasinya dengan mempelajari contoh script, memperbaiki bagian yang salah, dan mencoba menjalankan pengujian kembali sampai semua test berhasil.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda `[...]` sudah diganti dengan data API saya	YA
2	Setiap endpoint API saya ada di Collection	YA
3	Alamat memakai `{{base_url}}`, bukan ditulis langsung	YA
4	Setiap request punya assertion status code dan struktur JSON	YA
5	Nama setiap test berupa kalimat yang menyatakan harapan	YA
6	POST menyimpan ID dengan pemeriksaan status, dan request berikutnya memakainya	YA
7	Server dinyalakan ulang sebelum Runner dijalankan	YA
8	Screenshot Runner menampilkan 0 test gagal dan angkanya terbaca jelas	YA
9	Berkas Environment tidak memuat kunci rahasia sungguhan	YA
10	Berkas disimpan sebagai PDF dan dikumpulkan sesuai ketentuan LMS	YA

Lampiran B Kriteria Penilaian

Kriteria berikut sesuai dengan slide materi Pertemuan 8. Bobot nilai mengikuti ketentuan dosen di LMS.

Kriteria	Deskripsi	Dinilai dari
Kelengkapan Postman Collection	Seluruh endpoint project akhir tercakup dalam satu Collection	Bab 1 dan Bab 2
Ketepatan automated test script	Assertion status code dan struktur response JSON tepat sasaran	Bab 3
Keberhasilan Collection Runner	Seluruh test lolos saat Collection dijalankan	Bab 4
Kejelasan dokumentasi hasil	Tangkapan layar ringkasan pengujian disertakan dengan jelas	Bab 4 dan seluruh dokumen